# Project Phantasm: Real-Time Rendering of a Desaturated Horror Environment

Valean Sonia, Group 30435

January 2026

# Contents

**1 Introduction** **3**
1.1 Context . . . . . 3
1.2 Problem and Motivation . . . . . 3

**2 Bibliographic Study and Algorithms** **3**
2.1 Modified Blinn-Phong Lighting Model . . . . . 3
2.2 Shadow Mapping Algorithm . . . . . 3

**3 Analysis and Design** **4**
3.1 System Structure . . . . . 4

**4 Implementation** **4**
4.1 Geometric Data Management . . . . . 4
4.2 Fog System (Fog Algorithm) . . . . . 5
4.3 Lighting and Shadow Mapping in Shader . . . . . 5

**5 Interaction Details and Animation** **5**

**6 Testing and Validation** **6**
6.1 Test Case 1: Navigation and Camera . . . . . 6
6.2 Test Case 2: Shadow Mapping and Bias . . . . . 6
6.3 Test Case 3: Wireframe Mode . . . . . 7
6.4 Test Case 4: Atmosphere and Fog . . . . . 8

**7 Conclusions** **8**

# 1 Introduction

## 1.1 Context

Modern real-time rendering often focuses on vibrant and saturated environments. However, in the horror genre, the absence of color is a powerful narrative tool. This project explores the technical implementation of an abandoned and "time-bleached" amusement park. By utilizing advanced lighting techniques in a nocturnal setting, we simulate how moonlight and local artificial sources interact with oxidized and faded surfaces.

## 1.2 Problem and Motivation

The challenge lies in rendering a high-density environment (complex objects such as a carousel, stalls, mountains) while maintaining a high frame rate. The artistic motivation is to create a "bleached" nightmare atmosphere, where the contrast between the red fog and the cold blue moonlight creates a visual dissonance specific to the surrealist style.

# 2 Bibliographic Study and Algorithms

## 2.1 Modified Blinn-Phong Lighting Model

Instead of a simple model, I implemented a variant that combines directional light (the moon) with multiple point sources (the park lamps). The formula used in the fragment shader for each light is:

$$I = I_{amb} + \text{att} \times (I_{diff} + I_{spec}) \tag{1}$$

where the attenuation (att) is calculated based on the distance $d$ from the fragment:

$$\text{attenuation} = \frac{1.0}{1.0 + 0.09 \cdot d + 0.032 \cdot d^2} \tag{2}$$

## 2.2 Shadow Mapping Algorithm

Shadow mapping is a two-step process implemented through the `ShadowMapper` class: 1. **Pass 1 (Depth Pass):** The scene is rendered from the perspective

of the directional light into a special Framebuffer (FBO). 2. **Pass 2 (Lighting Pass):** In the main shader, the fragment's coordinates are transformed into light space to compare the current depth with the one stored in the *shadow map*.

# 3 Analysis and Design

## 3.1 System Structure

The system is organized modularly:

- **SceneManager:** Manages the object lifecycle, initializes the camera and lights.
- **ModelLoader:** Uses the Assimp library to load complex geometries and textures.
- **Camera:** "Free-look" navigation system with rotation around a target point.
- **InputHandler:** Captures keyboard and mouse events for interaction.

# 4 Implementation

## 4.1 Geometric Data Management

Each object in the scene is stored in a `GameObject` that encapsulates a **VAO** (Vertex Array Object). Data is sent to the GPU using `VBO` for positions, normals, and UV coordinates.

Listing 1: Fragment from ModelLoader for VAO configuration

```
void Mesh::setupMesh() {
    glGenVertexArrays(1, &VAO);
    glGenBuffers(1, &VBO);
    glGenBuffers(1, &EBO);
    glBindVertexArray(VAO);
    // ... attribute configuration (position, normal, texCoords)
    glEnableVertexAttribArray(0);
```

```
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, sizeof(Vertex),
        (void*)0);
}
```

### 4.2 Fog System (Fog Algorithm)

The red fog is calculated in screen space based on the distance from the camera to the fragment. I used an exponential function for a more realistic dispersion:

```
float calculateFog(float fogDensity) {
    float distance = length(viewPos - fragmentPos);
    float fogFactor = exp(-pow(distance * fogDensity, 2.0));
    return clamp(fogFactor, 0.0, 1.0);
}
```

### 4.3 Lighting and Shadow Mapping in Shader

To avoid the "Shadow Acne" effect, I implemented a variable bias based on the light's angle of incidence:

```
float shadow = currentDepth - bias > closestDepth ? 1.0 : 0.0;
```

## 5 Interaction Details and Animation

- **Navigation:** The user can move using the **W, A, S, D** keys and **SPACE/CTRL** for altitude. Rotation is performed by holding **Right Click** and moving the mouse.
- **Wireframe Visualization:** Pressing the **Q** key toggles between *Solid* and *Wireframe* mode through the `glPolygonMode` function.
- **Animation:** The carousel and park elements are animated by applying an incremental rotation matrix in the main loop: `model = glm::rotate(model, time, vec3(0,1,0))`.

# 6 Testing and Validation

This section presents the results obtained by running the application under different conditions.

## 6.1 Test Case 1: Navigation and Camera

**Objective:** Verifying the fluidity of movement and the correctness of the perspective projection.

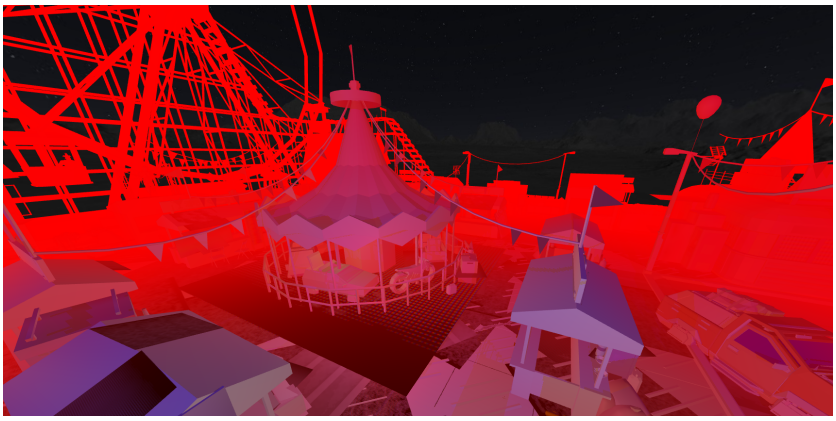

Figure 1: SCENE VIEWED FROM A DISTANCE

**Observation:** The camera behaves correctly, and the glm::perspective matrix maintains the window's aspect ratio.

## 6.2 Test Case 2: Shadow Mapping and Bias

**Objective:** Observing shadows on white surfaces.

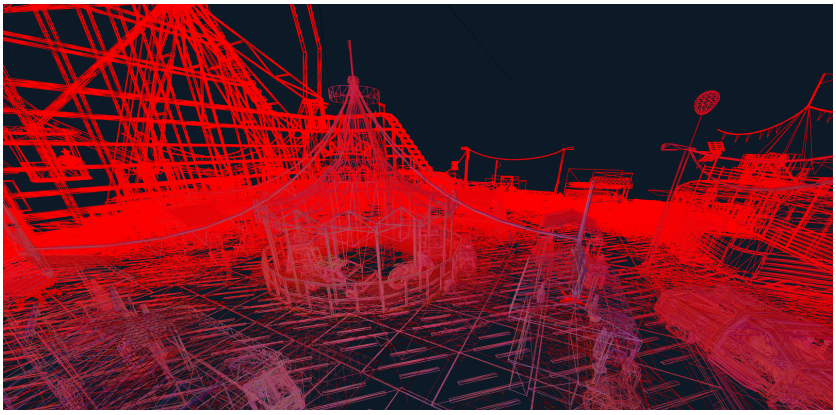

Figure 2: SCENE VIEWED FROM A DISTANCE

*[CAROUSEL SHADOW ON THE GROUND]*
**Observation:** Shadows are correctly projected from the moonlight source (0, 5, 20). The bias eliminates "banding" artifacts.

## 6.3 Test Case 3: Wireframe Mode

**Objective:** Validating the polygonal density of the GLTF models.

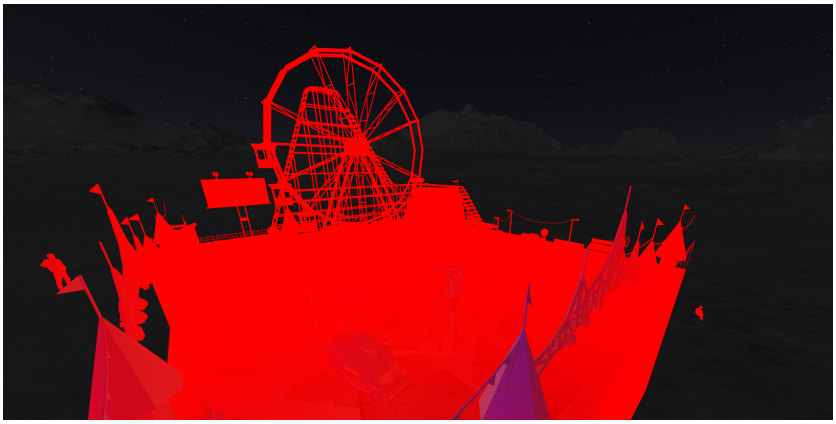

Figure 3: SCENE VIEWED FROM A DISTANCE

*[ WIREFRAME MODE ACTIVATED - Q KEY]*
**Observation:** The complex structure of the carousel is visible, demonstrating the efficiency of rendering a large number of triangles.

### 6.4 Test Case 4: Atmosphere and Fog

**Objective:** Testing the blending between colors and the red fog.

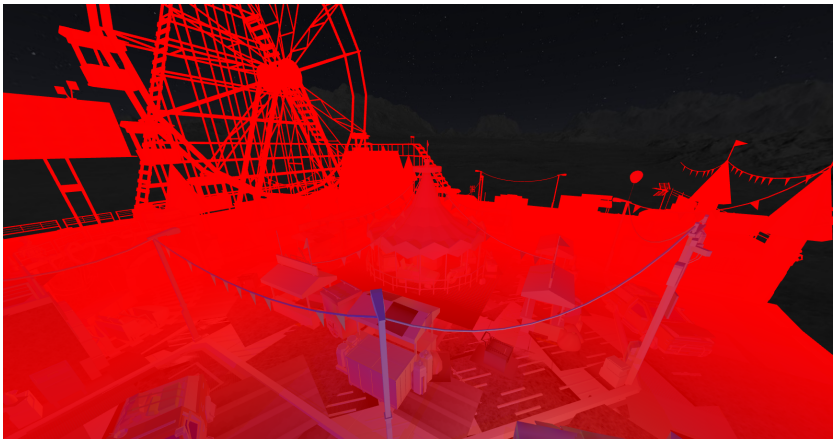

Figure 4: SCENE VIEWED FROM A DISTANCE

*[OBJECTS IN THE DISTANCE FADING INTO RED]*
**Observation:** Objects at more than 50 units away become completely red, according to the set density.

## 7 Conclusions

Project **Phantasm** demonstrates that a limited color palette — white, blue, and red — can create a horror atmosphere much more effectively than a full-color render. Technically, the use of textures, dynamic shadows, and the volumetric fog system provides the necessary realism, while memory optimizations ensure smooth performance.